GUION ULTRADETALLADO

Exposición de pruebas de software

Jeff · Sam · Mel

ALCANCE VERIFICADO

Documento alineado con el código ejecutable del backend al 17 de julio de 2026. Incluye una caja blanca, una caja negra y cuatro funciones/casos unitarios oficiales por expositor, más evidencia real de Bun, TypeScript y Stryker.

Expositor	Caja blanca	Caja negra	Unitarias oficiales
JEFF	HU01 AuthService.login	HU18 validateCreateAdvising	HU20 validateResetToken · validateNewPassword · maskEmail · hashOtp
SAM	HU30 notifyStudents	HU22 getAttendanceRisk	HU22/HU06 classifyStudent · splitName · computeSummary · calcularPromedioPonderado
MEL	HU25 updateMine	HU25 updateNetworkingSchema	HU25 validateSocialLink · urlBelongsToPlatform · validateNetworkingSelection · normalizeSocialLink

Proyecto ULima++ · Release 2 · Backend TypeScript/Bun

Preparado para compartir y ensayar en equipo

Cómo usar este documento

No intenten memorizar todo. Cada sección está pensada en tres niveles:

1. Ficha rápida: qué archivo abrir, qué función señalar y cuál es la frase central.

2. Guion largo: explicación paso a paso, tablas y texto literal para una exposición de 8–10 minutos.

3. Modo emergencia: resumen de 2 minutos y respuestas breves si el docente interrumpe.

MARCA EN EL CÓDIGO

Los diez archivos de prueba tienen comentarios pedagógicos y marcas . En VS Code, busquen ‘ ’ para saltar directamente a los puntos que conviene enseñar.

Índice automático

En Word: clic derecho aquí → Actualizar campo → Actualizar toda la tabla.

Mapa de cumplimiento y evidencia real

Persona	Tipo	Archivo de prueba	Conteo verificado	Evidencia especial
Jeff	Caja blanca	test/HU01_jeff/ login.cajablanca.test.ts	8/8	V(G)≈8 · Stryker 47/47
Jeff	Caja negra	test/HU18_jeff/ advising.cajanegra.test.ts	15/15	6 campos · particiones + bordes
Jeff	Unitarias	test/HU20_jeff/password-reset.logic.test.ts	14/14 total; 12 de las 4 funciones	generateOtp es respaldo
Sam	Caja blanca	test/HU30_sam/ notificar.cajablanca.test.ts	10/10	V(G)=10
Sam	Caja negra	test/HU22_sam/ impedidos.cajanegra.test.ts	6/6	6 campos · 6 particiones
Sam	Unitarias	HU22 impedidos.unitarias + HU06 grades.logic	4 casos oficiales; suites más amplias	Una prueba por función pedida
Mel	Caja blanca	test/HU25_mel/ networking.cajablanca.test.ts	5/5	V(G)=5 · Stryker 15/15
Mel	Caja negra	test/HU25_mel/ networking.cajanegra.test.ts	8/8 bloques	12 payloads · 13 aserciones
Mel	Unitarias	test/HU25_mel/ networking.logic.test.ts	9/9 total	4 funciones oficiales + isHttpUrl de apoyo

Resultado de verificación ejecutado

Control	Resultado	Interpretación correcta
10 archivos seleccionados con Bun	98 pass · 0 fail · 2201 expect	Todas las suites de la exposición están verdes
Compilación TypeScript	bun run build → tsc sin errores	Los comentarios no rompieron tipos ni sintaxis
Stryker Jeff	47 killed · 0 survived	Solo auth.service.ts:56–134
Stryker Sam	156 killed · 0 survived · 2 ignored	147 asistencia + 9 notas; configuración combinada
Stryker Mel	15 killed · 0 survived	Solo networking.service.ts:31–56

NO CONFUNDIR

100% de mutación no significa 100% de cobertura, ausencia de bugs ni calidad de todo el sistema. Significa que, dentro del rango mutado, todos los mutantes válidos fueron detectados por las pruebas configuradas.

Conceptos que los tres deben dominar

Caja blanca

Diseña casos conociendo decisiones, ramas, bucles, excepciones y flujo interno. La evidencia típica es un grafo de control y una base de caminos independientes.

Fórmulas de McCabe

```
V(G) = número de decisiones + 1
V(G) = E − N + 2P   (E=aristas, N=nodos, P=componentes conectados)
```

La complejidad indica un mínimo teórico de caminos independientes para la estructura modelada; no demuestra por sí sola que el comportamiento sea correcto.

Caja negra

Diseña entradas desde requisitos y contrato observable. Se agrupan datos equivalentes y se eligen representantes válidos, inválidos y fronteras.

- Partición de equivalencia: varios valores que deberían producir la misma clase de resultado.
- Valor límite: punto exacto donde cambia la regla, más valores inmediatamente adyacentes cuando corresponde.
- Oráculo: resultado observable esperado; por ejemplo status, success, error o dato normalizado.

Prueba unitaria y dobles de prueba

Una prueba unitaria aísla una función o unidad lógica. No necesita que la función sea de una sola línea; necesita controlar sus dependencias externas.

Doble	Uso en este proyecto	Ejemplo
Dummy	Objeto requerido pero no utilizado	EventBus vacío
Stub	Devuelve datos predeterminados	Repositorio que entrega una fila preparada
Spy	Captura llamadas y argumentos	captured[] en notifyStudents
Fixture	Objeto reutilizable con valores base	student(), nrow(), row(), input()

Patrón AAA

Fase	Qué ocurre	Pregunta para explicarlo
Arrange	Se crean fixtures, stub/spy y servicio	¿Qué estado obliga el camino?
Act	Se invoca el método real	¿Cuál es la única acción bajo prueba?
Assert	Se compara salida y/o interacción	¿Qué observable demostraría que la regla funciona?

Testing de mutación

1. Stryker modifica temporalmente el código: cambia operadores, condiciones, textos o retornos.

2. Ejecuta la suite configurada contra cada cambio artificial.

3. Si una prueba falla, el mutante queda killed: la suite detectó la alteración.

4. Si todo sigue verde, el mutante sobrevive: puede faltar una aserción o ser un mutante equivalente.

FRASE COMÚN SEGURA

‘La prueba de mutación evalúa la sensibilidad de nuestra suite ante cambios artificiales del código. El porcentaje siempre se interpreta dentro del rango configurado.’

JEFF

HU01 caja blanca · HU18 caja negra · HU20 cuatro funciones unitarias

Ficha rápida de Jeff

Elemento	Qué abrir	Qué demostrar
Caja blanca	HU01_jeff/login.cajablanca.test.ts	8 recorridos: alumno, docente, matrícula, rol y error 500
Caja negra	HU18_jeff/advising.cajanegra.test.ts	6 campos y 15 casos por partición, borde y precedencia
4 unitarias	HU20_jeff/password-reset.logic.test.ts	Token, contraseña, correo enmascarado y hash del OTP
Mutación	bun run mut:jeff	47/47 mutantes de auth.service.ts:56–134

TEXTO SUGERIDO PARA DECIR

“Soy Jeff. Elegí el login de la HU01 para caja blanca porque concentra decisiones de autenticación, matrícula, rol y manejo de errores. Para caja negra presento la validación de asesorías de la HU18, y mis cuatro unidades puras pertenecen al restablecimiento de contraseña de la HU20.”

Orden recomendado y tiempo

Minuto	Acción	Objetivo
0:00–0:40	Presentación y mapa de archivos	Que el jurado entienda la asignación
0:40–4:30	HU01: código, grafo, caminos y 2 casos	Defender caja blanca y V(G)
4:30–6:40	HU18: campos, particiones y bordes	Defender caja negra
6:40–8:20	HU20: cuatro funciones	Defender aislamiento y seguridad
8:20–9:20	Bun + Stryker	Mostrar evidencia
9:20–10:00	Cierre	Resumir sin sobreafirmar

1. Caja blanca de Jeff — AuthService.login()

Producción: src/modules/auth/auth.service.ts — login():56–100; loginTeacher():107–131
Prueba: test/HU01_jeff/login.cajablanca.test.ts
Entrada HTTP: POST /login → auth.routes.ts → auth.controller.ts → AuthService.login

IDEA MÁS IMPORTANTE

Se ejecuta AuthService real. El repositorio es falso para forzar caminos deterministas, mientras bcrypt y JWT se ejecutan de verdad. La suite conoce la estructura interna: por eso es caja blanca.

Flujo del código de producción

1. Busca un alumno por code mediante findByCodeWithPassword.
2. Si no existe alumno, delega a loginTeacher; ahí verifica existencia y contraseña del docente.
3. Si existe alumno, compara password con passwordHash usando bcrypt.compare.
4. Exige matrícula activa; si no la hay, devuelve 403 NOT_ENROLLED.
5. Consulta representación activa y usa position o el rol student por defecto.
6. Incrementa tokenVersion para que el JWT nuevo tenga una versión de sesión actualizada.
7. Elimina passwordHash del objeto público y firma el JWT con studentId o teacherId según corresponda.
8. Repropaga HttpError de negocio; cualquier Error inesperado se registra y se convierte en 500 genérico.

Extracto conceptual que conviene señalar

```
const user = await repository.findByCodeWithPassword(input.code);
if (!user) return await this.loginTeacher(input);

const matches = await bcrypt.compare(input.password, user.passwordHash);
if (!matches) throw new HttpError(401, ..., 'INVALID_PASSWORD');
if (!await repository.hasActiveEnrollment(user.studentId))
  throw new HttpError(403, ..., 'NOT_ENROLLED');

const role = representation?.position ?? 'student';
```

Complejidad ciclomática

El grafo documentado tiene 21 nodos y 27 aristas en un componente:

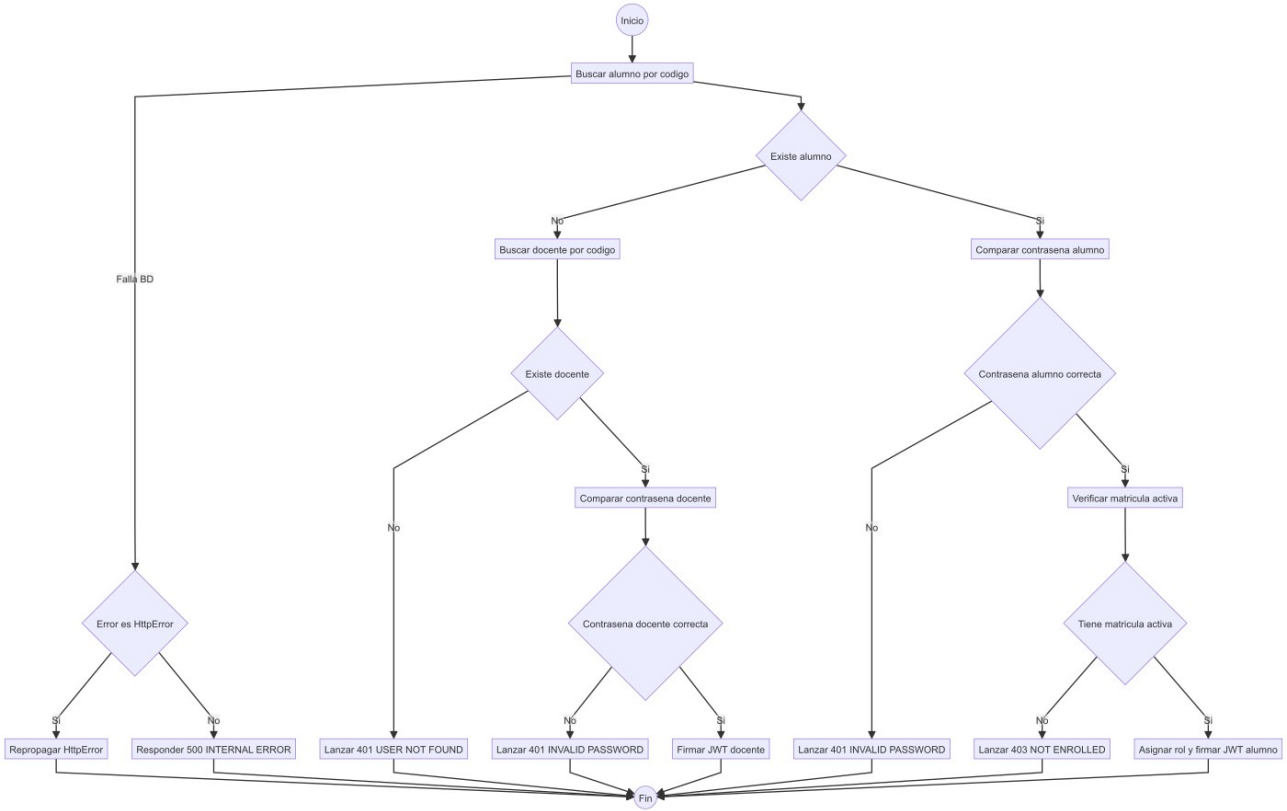
```
V(G) = E − N + 2 = 27 − 21 + 2 = 8
```

También puede defenderse como aproximadamente ocho regiones bajo la convención que modela el catch/selección de rol. Algunas herramientas podrían dar siete si tratan ?? o excepciones de otra manera; por eso la frase segura es ‘V(G)≈8 y ocho recorridos observables’.

NO DIGAS

‘Tiene ocho decisiones’. El modelo tiene decisiones y una región base; la afirmación precisa es ‘complejidad aproximada ocho’.

Grafo de flujo



Grafo de AuthService.login/loginTeacher. El nodo de asignación de rol resume representation?.position ?? 'student'.

Archivo Mermaid editable: Guion_Pruebas_Jeff_Sam_Mel/mermaid/grafo_jeff_auth_login.mmd

Ocho caminos independientes

Camino	Recorrido forzado	Oráculo observable
C1	No hay alumno → docente existe → contraseña correcta	JWT docente; teacherId=7; sin studentId; hash oculto
C2	No hay alumno ni docente	401 USER_NOT_FOUND
C3	Docente existe; contraseña incorrecta	401 INVALID_PASSWORD
C4	Alumno existe; contraseña incorrecta	401 INVALID_PASSWORD

Camino	Recorrido forzado	Oráculo observable
C5	Alumno correcto; matrícula inactiva	403 NOT_ENROLLED
C6	Alumno correcto; representación delegate	JWT alumno con role=delegate y tokenVersion nuevo
C7	Alumno correcto; sin representación	Role=student por valor por defecto
C8	El repositorio lanza Error no-HttpError	Log interno + 500 INTERNAL_ERROR público

TEXTO SUGERIDO PARA DECIR

"El caso C1 demuestra el camino docente completo y C6/C7 contrastan el rol del alumno con y sin representación. C8 es especialmente importante porque demuestra que un fallo interno no filtra detalles: se registra en servidor y el cliente solo recibe INTERNAL_ERROR."

Cómo está construido el test

- PASSWORD y HASH: bcrypt produce un hash real; no se falsea la comparación criptográfica.
- student() y teacher(): fixtures frescos para evitar estado compartido entre tests.
- makeService(options): stub del repositorio; cada opción activa una rama concreta.
- calls.incrementedUserIds y enrollmentStudentIds: espías manuales del flujo de datos.
- claimsFrom(): verifica la firma JWT con el mismo secreto y permite inspeccionar claims.
- expectHttpError(): exige statusCode, code y message exactos.
- spyOn(console, 'error') en C8: silencia y verifica el log; finally restaura el spy.

AAA del camino C6

```
// Arrange
const { service, calls } = makeService({ student: student(), representation: { position: 'delegate' } });
// Act
const result = await service.login({ code: '20239901', password: PASSWORD });
// Assert
expect(claimsFrom(result.token).role).toBe('delegate');
expect(calls.incrementedUserIds).toEqual([11]);
```

Mutación de Jeff

```
bun run mut:jeff
# muta: src/modules/auth/auth.service.ts:56-134
# oráculo: test/HU01_jeff/login.cajablanca.test.ts
```

Métrica	Resultado
Mutantes instrumentados	47
Killed	47
Survived	0
No coverage	0
Mutation score	100% dentro del rango

TEXTO SUGERIDO PARA DECIR

"Stryker generó 47 cambios artificiales únicamente dentro del rango de login y loginTeacher. La caja blanca detectó los 47. Esto demuestra sensibilidad de la suite en ese rango, no ausencia total de defectos."

2. Caja negra de Jeff — validateCreateAdvising()

Producción: src/modules/advising/teacher/teacher.logic.ts — validateCreateAdvising():77–99

Prueba: test/HU18_jeff/advising.cajanegra.test.ts

ALCANCE EXACTO

Es caja negra de la frontera de la función de validación. No prueba HTTP, autenticación, ownership ni base de datos. Los seis campos son los consumidos por CreateAdvisingInput; el payload HTTP completo también tiene sectionId, note y capacity.

Seis campos de entrada

Campo	Clase válida	Clase inválida / frontera	Salida relacionada
sessionDate	today ≤ fecha ≤ periodEnd	pasado; fuera del periodo; bordes inclusivos	date_in_past / date_out_of_period / ok
startTime	Menor que endTime	Mayor o igual a endTime	invalid_time
endTime	Mayor que startTime	Menor o igual a startTime	invalid_time
modality	classroom / virtual / hybrid	La capa Zod rechaza otros valores	Define ubicación requerida
classroom	Texto no vacío si presencial	vacío o espacios	missing_location
meetingUrl	Texto no vacío si virtual	vacío o ausente	missing_location

Orden observable de precedencia: invalid_time → date_in_past → date_out_of_period → missing_location → overlap. Los retornos tempranos hacen que el primer error gane.

Tabla de 15 casos

ID	Entrada representativa	Esperado	Técnica
CV1	6 campos base válidos	ok + dayOfWeek	Válida
CNV1	12:00 → 10:00	invalid_time	Partición inválida
CNV2	10:00 → 10:00	invalid_time	Límite: duración cero
CNV3	sessionDate anterior a today	date_in_past	Partición inválida
CV2	sessionDate == today	ok	Límite inclusivo
CNV4	sessionDate posterior a periodEnd	date_out_of_period	Partición inválida
CV3	sessionDate == periodEnd	ok	Límite inclusivo
CNV5	classroom sin aula	missing_location	Partición inválida
CNV6	classroom con aula=' '	missing_location	Límite por trim
CV4	virtual con meetingUrl no vacío	ok	Válida
CNV7	virtual sin meetingUrl	missing_location	Partición inválida
CNV8	10–12 se cruza con 11–13	overlap	Partición inválida
CV5	08–10 toca 10–12	ok	Límite sin solape
P1	Horario inválido + fecha pasada	invalid_time	Precedencia
P2	Sin aula + solape	missing_location	Precedencia

TEXTO SUGERIDO PARA DECIR

“La fecha today se inyecta como un valor fijo para que el caso no cambie según el día. Los bordes que solo se tocan no solapan porque la comparación usa desigualdades estrictas; en cambio, 10–12 con 11–13 sí comparte tiempo y devuelve overlap.”

Contrato observable y precedencia

```
if (!isValidTimeRange(...)) return { status: 'invalid_time' };
if (isDateInPast(...)) return { status: 'date_in_past' };
if (!isDateWithinPeriod(...)) return { status: 'date_out_of_period' };
if (!hasRequiredLocation(...)) return { status: 'missing_location' };
if (overlaps) return { status: 'overlap' };
return { status: 'ok', dayOfWeek };
```

PRECISIÓN

meetingUrl solo se exige como texto no vacío en esta función. No afirmes que esta suite valida una URL real; ese formato pertenece a otra capa y el schema actual tampoco usa .url() aquí.

3. Cuatro unitarias de Jeff — HU20

Producción: src/modules/auth/password-reset.logic.ts

Prueba: test/HU20_jeff/password-reset.logic.test.ts

Las cuatro unidades oficiales son funciones puras: reciben datos y retornan resultados sin tocar base de datos, reloj global ni correo. El archivo tiene además generateOtp como cobertura de respaldo.

Unidad oficial	Qué prueba	Casos	Punto clave para decir
1. validateResetToken	Estados y precedencia del token	7	usado → expirado → intentos → hash → ok
2. validateNewPassword	Longitud mínima	2	7 rechaza; 8 acepta
3. maskEmail	Minimización de datos	2	muestra 4 o al menos 1 carácter; conserva dominio
4. hashOtp	SHA-256 determinístico	1	64 hex; vector conocido; otro OTP cambia

validateResetToken — la unidad principal

```
if (usedAt != null) return already_used;
if (now >= expiresAt) return expired;
if (attempts >= 5) return too_many_attempts;
const matches = sameLength && timingSafeEqual(candidateHash, storedHash);
return matches ? ok : mismatch;
```

Caso	Cambio respecto al token sano	Esperado
Feliz	OTP correcto; vigente; sin uso; attempts=0	ok
Expirado posterior	expiresAt está en el pasado	expired
Frontera	now == expiresAt	expired
Usado	usedAt no es null	already_used
Máximo	attempts == 5	too_many_attempts
Uno antes	attempts == 4	ok
No coincide	candidateOtp distinto	mismatch

TEXTO SUGERIDO PARA DECIR

“Paso now como parámetro para que el límite sea determinista. En el instante exacto de vencimiento ya se considera expirado porque se usa mayor o igual. La comparación de hashes usa timingSafeEqual y primero comprueba que las longitudes coincidan.”

Las otras tres unidades

- validateNewPassword: demuestra solo la política actual de mínimo ocho caracteres; no demuestra entropía ni fortaleza completa.
- maskEmail: evita mostrar el correo institucional completo; la rama de correo inválido existe en producción, aunque no está entre los dos casos actuales.
- hashOtp: fija SHA-256 con un vector conocido. No debe decirse que un único test demuestra ausencia de colisiones.
- generateOtp: dos pruebas adicionales verifican seis dígitos y variedad; queda de respaldo porque una propiedad de variedad es probabilística.

SEGURIDAD QUE CONVIENE ACLARAR

El OTP temporal se compara mediante un hash SHA-256 determinístico; la contraseña persistente se almacena con bcrypt. No son intercambiables ni cumplen el mismo propósito.

Preguntas probables para Jeff

Pregunta	Respuesta breve y segura
¿Por qué repo falso?	Para controlar cada rama y evitar que la BD vuelva inestable la prueba.
¿bcrypt y JWT están mockeados?	No; se ejecutan realmente. Solo se sustituye persistencia.
¿Por qué 401 vs 403?	401 para credenciales inválidas; 403 para usuario identificado sin matrícula activa.
¿Qué hace tokenVersion?	Entra en el JWT y permite invalidar versiones de sesión anteriores.
¿Por qué probar docentes en HU01?	login() delega internamente a loginTeacher; omitirlo dejaría caminos reales sin recorrer.
¿Por qué fijar today?	Evita que una fecha cambie de clase entre ejecuciones.
¿8 caminos significa 8 bugs?	No; son recorridos independientes del grafo, no defectos.
¿100% mutación garantiza todo?	No; solo detectó 47 mutantes válidos en el rango configurado.

Demo de Jeff

```
cd ~/ULIMA++/ULima_Backend_IS2

bun test test/HU01_jeff/login.cajablanca.test.ts
bun test test/HU18_jeff/advising.cajanegra.test.ts
bun test test/HU20_jeff/password-reset.logic.test.ts
bun run mut:jeff
```

PLAN B SI NO HAY TIEMPO

Ejecuta primero la caja blanca y luego Stryker. Las otras dos suites ya pueden mostrarse en el terminal o en este documento.

Versión de 2 minutos de Jeff

TEXTO SUGERIDO PARA DECIR

“Mi caja blanca es AuthService.login. Aíslo el repositorio, pero ejecuto bcrypt y JWT reales. Ocho casos fuerzan los recorridos de docente, usuario inexistente, contraseñas inválidas, matrícula, representación y error interno; el grafo da V(G) aproximadamente 8 y Stryker elimina 47 de 47 mutantes del rango. Mi caja negra es validateCreateAdvising: seis campos y quince casos de equivalencia, fronteras, solape y precedencia. Finalmente, mis cuatro unidades puras de HU20 validan el token de reset, la longitud de la nueva contraseña, el enmascarado del correo y el hash SHA-256 del OTP.”

Errores que Jeff debe evitar

- Decir que el login toca una base de datos real durante el test.
- Llamar cobertura de código al 47/47 de mutación.
- Afirmar que HU18 tiene solo seis campos HTTP; son seis campos de la función seleccionada.
- Afirmar que meetingUrl se valida como URL real en esta suite.
- Decir que hay solo cuatro bloques unitarios: hay cuatro funciones oficiales y doce casos entre ellas.

SAM

HU30 caja blanca · HU22 caja negra · HU22/HU06 cuatro unitarias

Ficha rápida de Sam

Elemento	Qué abrir	Qué demostrar
Caja blanca	HU30_sam/notificar.cajablanca.test.ts	V(G)=10; selección y persistencia exacta de alertas
Caja negra	HU22_sam/impedidos.cajanegra.test.ts	6 campos; 25%/35%; fronteras y cero horas
4 unitarias	HU22 impedidos.unitarias + HU06 grades.logic	Una prueba por classifyStudent, splitName, computeSummary y promedio
Mutación	bun run mut:sam	147 asistencia + 9 notas; 2 equivalentes ignorados

TEXTO SUGERIDO PARA DECIR

“Soy Sam. Mi caja blanca analiza notifyStudents de la HU30, donde el servicio recorre alumnos y decide qué alertas persistir. Mi caja negra clasifica riesgo de asistencia en la HU22. Mis cuatro pruebas unitarias oficiales cubren clasificación, separación de nombres, resumen y promedio ponderado.”

Orden recomendado y tiempo

Minuto	Acción	Objetivo
0:00–0:40	Presentación	Ubicar HU30, HU22 y HU06
0:40–4:40	notifyStudents + grafo + spy	Defender V(G)=10 y flujo de datos
4:40–6:40	Tabla negra HU22	Defender 6 inputs y fronteras
6:40–8:20	4 unitarias	Una función por caso oficial
8:20–9:20	Bun + mutación	147 asistencia / 156 total puntuable
9:20–10:00	Cierre	Resultados y limitaciones

1. Caja blanca de Sam — notifyStudents()

Producción: src/modules/attendance-risk/attendance-risk.service.ts:152–193
Prueba: test/HU30_sam/notificar.cajablanca.test.ts
Endpoint: POST /sections/:sectionId/attendance-risk/notify — rol teacher

IDEA MÁS IMPORTANTE

La prueba inspecciona decisiones internas y captura el arreglo exacto enviado a createAlerts. Ese spy demuestra qué alumnos se persisten y cuáles se descartan.

Flujo del código

1. Obtiene filas con detalles de los alumnos y fija sessionHours=2.
2. Recorre cada fila; si total_section_hours≤0, continúa sin alerta.
3. Calcula porcentaje y elige límite 25% para ciclos 1–5 o 35% para ciclo 6+.
4. Si porcentaje>límite, redacta la alerta de impedido.
5. Si no, calcula faltas restantes con ceil((máximo−ausentes)/2).
6. Solo si faltan exactamente 2 o 3 crea una alerta preventiva; los demás valores continúan.
7. Agrega alertas al arreglo y llama a createAlerts una sola vez con ese arreglo.
8. Construye mensaje de cero, singular o plural según notified.

Decisiones que originan el grafo

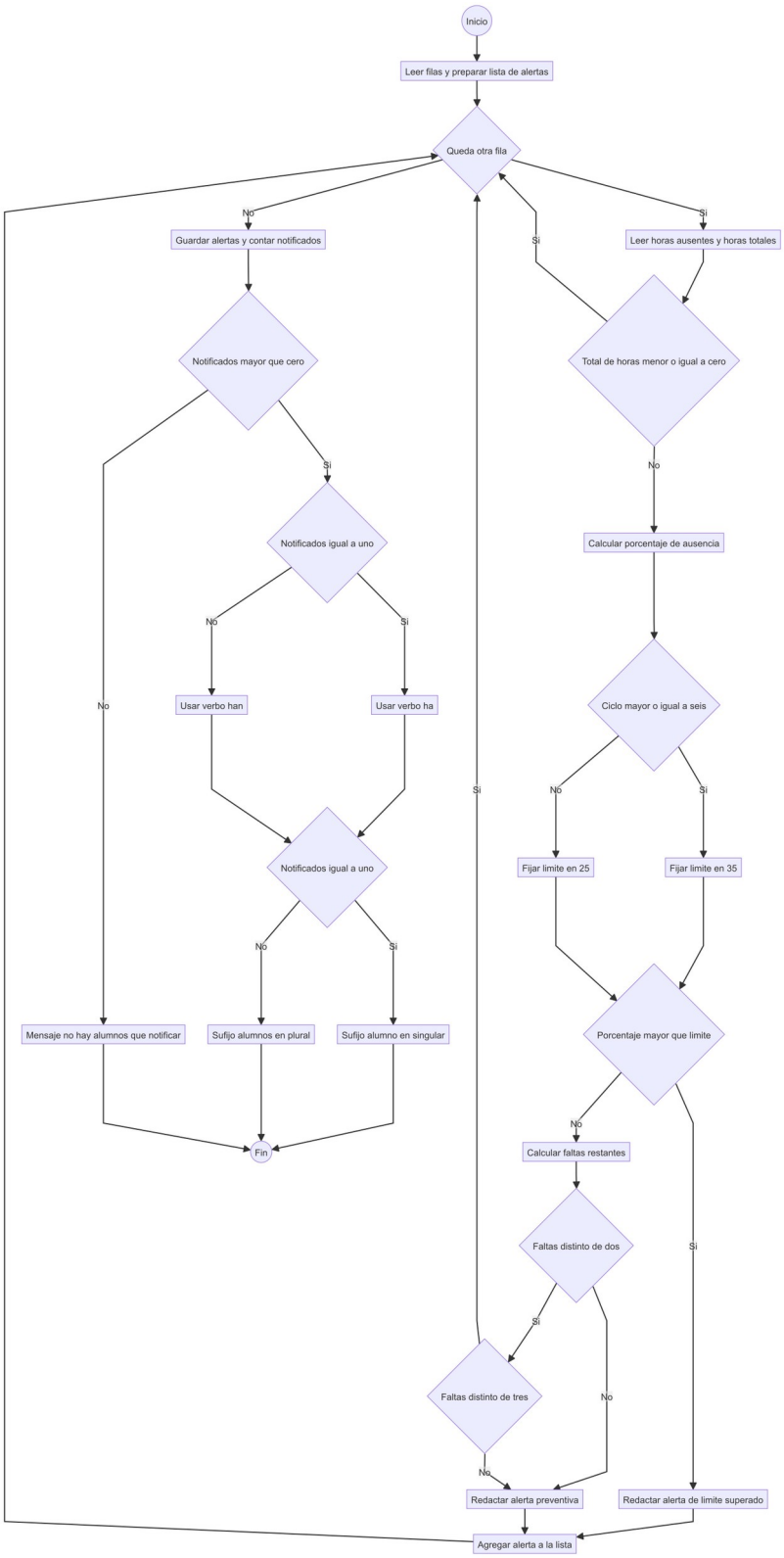
```
for (const row of rows) {
  if (totalSectionHours <= 0) continue;
  const limit = cycle >= 6 ? 35 : 25;
  if (absencePercentage > limit) { /* impedido */ }
  else {
    const faltas = Math.ceil(remaining / 2);
    if (faltas !== 2 && faltas !== 3) continue;
    /* preventiva */
  }
  alerts.push(...);
}
const notified = await repository.createAlerts(alerts);
```

V(G)=10, conteo defendible

#	Decisión
1	for: queda otra fila
2	totalSectionHours ≤ 0
3	cycle ≥ 6
4	absencePercentage > limit
5	faltas !== 2
6	faltas !== 3
7	notified > 0
8	notified === 1 para el verbo
9	notified === 1 para alumno(s)

```
V(G) = 9 decisiones + 1 región base = 10
Comprobación del grafo: E − N + 2 = 34 − 26 + 2 = 10
```

Grafo de flujo



Grafo de notifyStudents. El activo histórico dice Aurelio en el nombre de archivo; la atribución actual de la exposición es Sam.

Archivo Mermaid editable renombrado: Guion_Pruebas_Jeff_Sam_Mel/mermaid/grafo_sam_notify_students.mmd

Diez caminos/casos

Caso	Entrada o rama	Resultado
C1	total_section_hours=0	continue; 0 alertas; ‘No hay alumnos que notificar.’
C2	30/100, ciclo 3	Alerta de límite superado (25%)

Caso	Entrada o rama	Resultado
C3	17/100, ciclo 3	A 4 faltas: no alerta
C4	21/100, ciclo 3	Preventiva: faltan 2 sesiones
C5	Sección mixta	Solo ids 1 y 2; plural correcto
C6	Un notificado	‘Se ha notificado a 1 alumno.’
C7	30/100, ciclo 6	Límite 35%; preventiva a 3 faltas
C8	25/100, ciclo 3	No alerta: comparación estricta >
C9	24/100, ciclo 3	A 1 falta: la implementación no notifica
C10	Filas válidas, todas normales	0 alertas tras evaluar datos reales

TEXTO SUGERIDO PARA DECIR

“El caso más completo es C5: entran cuatro filas, pero el spy demuestra que createAlerts recibe solo los ids 1 y 2. C8 muestra que el límite es estricto: 25% exacto todavía no lo supera. C7 usa el mismo 30% en ciclo 6 y demuestra el cambio del límite a 35%.”

Cómo funciona el spyService

Repositorio espía sin base de datos

```
const captured = [];
const repo = {
  findStudentDetailsBySectionId: async () => rows,
  createAlerts: async (data) => {
    captured.push(...data);
    return data.length;
  },
};
```

- rows controla el camino de entrada.
- captured observa la interacción que normalmente escribiría en la BD.
- return data.length simula el conteo que usa el servicio para singular/plural.
- El EventBus es un dummy porque este método no lo utiliza.

Mutación de Sam

```
bun run mut:sam
# muta archivos completos:
# - src/modules/attendance-risk/attendance-risk.service.ts
# - src/modules/grades/grades.logic.ts
# oráculo combinado: HU06, HU07, HU22 y HU30 de Sam
```

Archivo	Killed	Ignored	Lectura correcta
attendance-risk.service.ts	147	0	147/147; contribuyen varias suites Sam
grades.logic.ts	9	2	2 mutantes equivalentes de una guarda redundante
Total puntuable	156	2	100% de mutantes válidos

FRASE CORRECTA

‘La batería combinada de Sam elimina 147/147 mutantes de attendance-risk.service.ts. No atribuyo esos 147 exclusivamente al archivo HU30 porque la configuración ejecuta HU22 y otras suites también.’

2. Caja negra de Sam — getAttendanceRisk()

Producción: src/modules/attendance-risk/attendance-risk.service.ts:39–150

Prueba: test/HU22_sam/impedidos.cajanegra.test.ts

ALCANCE EXACTO

Es caja negra a nivel del servicio: el repositorio se sustituye y solo se observa la clasificación devuelta. No es una prueba HTTP completa.

Seis campos

Campo	Rol en la prueba
code	Identificador devuelto; se mantiene válido
full_name	Entrada para separación de nombres; se mantiene válida
current_level	Nivel mostrado; se mantiene válido
absent_hours	Varía el porcentaje y faltas restantes
total_section_hours	Denominador; incluye frontera cero
cycle	Selecciona límite 25% o 35%

Aunque solo tres campos varían entre particiones, la función recibe y procesa la fila completa de seis campos; los otros tres se mantienen como valores válidos de referencia.

Tabla de particiones

Caso	Entrada	Resultado esperado
CN1	30/100, ciclo 3	impedido; 30%; missingFaltas=null
CN2	30/100, ciclo 6	en_riesgo; missingFaltas=3
CN3	25/100, ciclo 3	normal: 25% no supera el límite
CN4	21/100, ciclo 3	en_riesgo; missingFaltas=2
CN5	17/100, ciclo 3	normal; a 4 faltas
CN6	4/0, ciclo 3	normal; 0%; sin NaN/división por cero

```
limit = cycle >= 6 ? 35 : 25
percentage = absent / total * 100
impedido ⇔ percentage > limit
remainingSessions = ceil((total*limit/100 − absent) / 2)
en_riesgo ⇔ remainingSessions ∈ {2,3}
```

TEXTO SUGERIDO PARA DECIR

“CN1 y CN2 usan el mismo 30%, pero el ciclo cambia el límite. CN3 prueba la frontera exacta de 25%, que sigue normal por la comparación estricta. CN6 protege la división por cero y exige 0%, no NaN.”

3. Cuatro unitarias de Sam

La captura pide cuatro funciones. Para coincidir literalmente, se presenta una prueba representativa por función; el resto de los archivos queda como cobertura complementaria.

#	Función	Prueba oficial	Resultado
1	classifyStudent	30/100, ciclo 6	en_riesgo; 3 faltas
2	splitName	‘Garcia Lopez, Maria’	lastName=Garcia Lopez; firstName=Maria
3	computeSummary	2 impedidos + 1 riesgo + 2 normales	{2,1,2,total:5}
4	calcularPromedioPonderado	12@30% + 16@50% + 8@20%	13.2

POR QUÉ SIGUEN SIENDO UNITARIAS

classifyStudent, splitName y computeSummary no se exportan. Se ejercitan a través de AttendanceRiskService con un repositorio stub; la unidad queda aislada de la base de datos.

Explicación de cada unidad

- classifyStudent: el mismo 30% cambia de impedido en ciclo 3 a en_riesgo en ciclo 6; defiende límite y Math.ceil.
- splitName: la coma separa apellidos de nombres y trim elimina espacios del borde.
- computeSummary: recorre los alumnos, incrementa uno de tres contadores y conserva total=students.length.
- calcularPromedioPonderado: función pura sin mocks; suma valor×peso/100 y usa toBeCloseTo por punto flotante.

Cálculo que Sam puede hacer en la pizarra

```
12 × 0.30 + 16 × 0.50 + 8 × 0.20
= 3.60 + 8.00 + 1.60
= 13.20
```

Preguntas probables para Sam

Pregunta	Respuesta breve y segura
¿Por qué Math.ceil?	Cinco horas restantes con sesiones de dos requieren tres sesiones, no dos.
¿Por qué horas como string?	El tipo del repositorio entrega valores numéricos de PostgreSQL como cadenas; el servicio usa Number.
¿Por qué 25% no es impedido?	La condición de producción es >, no >=.
¿Qué prueba el spy?	El arreglo exacto enviado a createAlerts y, por tanto, qué alumnos serían persistidos.
¿Por qué total cero es normal?	Es una guarda para evitar división por cero cuando aún no hay horas dictadas.
¿147/147 es solo HU30?	No; es attendance-risk.service con la batería combinada de Sam.
¿Por qué hay 2 ignored?	La guarda de lista vacía en notas es redundante: mutarla no cambia la salida; son equivalentes.

Demo de Sam

```
cd ~/ULIMA++/ULima_Backend_IS2

bun test test/HU30_sam/notificar.cajablanca.test.ts
bun test test/HU22_sam/impedidos.cajanegra.test.ts
bun test test/HU22_sam/impedidos.unitarias.test.ts test/HU06_sam/grades.logic.test.ts

# Solo los 4 casos oficiales
bun test test/HU22_sam/impedidos.unitarias.test.ts test/HU06_sam/grades.logic.test.ts \
  --test-name-pattern 'ciclo 6 con el MISMO|splitName con coma|2 impedidos \+ 1 en riesgo|ponderado de varias evaluaciones'

bun run mut:sam
```

Versión de 2 minutos de Sam

TEXTO SUGERIDO PARA DECIR

“Mi caja blanca es notifyStudents. Nueve decisiones más la región base dan V(G)=10; diez casos fuerzan cero horas, impedido, preventiva, límites de ciclo, frontera exacta y singular/plural. El repositorio espía captura qué alertas se guardarían. Mi caja negra de HU22 usa seis campos y seis particiones para clasificar impedido, en riesgo o normal, incluido total cero. Mis cuatro unidades oficiales representan classifyStudent, splitName, computeSummary y calcularPromedioPonderado. La batería combinada mata 147 de 147 mutantes de asistencia y 9 de notas; dos equivalentes se ignoran.”

Errores que Sam debe evitar

- Decir V(G)=8/9: el conteo declarado correcto es 10.
- Decir que CN2 es ciclo 3: el test real usa ciclo 6.
- Atribuir 147/147 únicamente a HU30.
- Decir que las cuatro unitarias son cuatro casos de classifyStudent; la selección oficial cubre cuatro funciones.

- Ocultar el comportamiento actual de una falta: la implementación no notifica; descríbelo como comportamiento observado si preguntan.

MEL

HU25 caja blanca · HU25 caja negra · HU25 cuatro funciones unitarias

Ficha rápida de Mel

Elemento	Qué abrir	Qué demostrar
Caja blanca	HU25_mel/networking.cajablanca.test.ts	V(G)=5 y cinco salidas de updateMine
Caja negra	HU25_mel/networking.cajanegra.test.ts	5 campos, esquema estricto, 8 bloques/12 payloads
4 unitarias	HU25_mel/networking.logic.test.ts	Etiqueta, dominio, cardinalidad y normalización
Mutación	bun run mut:mel	15/15 mutantes de updateMine

TEXTO SUGERIDO PARA DECIR

“Soy Mel. Mis tres bloques pertenecen a la HU25 de networking. La caja blanca estudia updateMine, la caja negra observa el contrato Zod y las cuatro unitarias principales protegen las reglas puras de enlace social.”

Orden recomendado y tiempo

Minuto	Acción	Objetivo
0:00–0:40	Presentación del endpoint	Explicar userId desde JWT
0:40–4:10	updateMine + V(G) + caminos	Defender caja blanca
4:10–6:40	Zod + tabla negra	Defender 5 campos y strict
6:40–8:20	4 funciones puras	Defender unitarias
8:20–9:20	Mutación	15/15 con alcance
9:20–10:00	Cierre	Seguridad y normalización

Contexto funcional de HU25

1. PUT /networking/me exige autenticación para roles permitidos.
2. El controlador toma userId del JWT; nunca confía en un userId del body.
3. updateNetworkingSchema valida el JSON antes de entrar al servicio.
4. updateMine aplica reglas de negocio, normaliza y llama a replaceByUserId.
5. El repositorio usa transacción, bloqueo FOR UPDATE, update, delete e insert.

SEGURIDAD CENTRAL

El propietario proviene del JWT. El schema .strict() rechaza userId enviado por el cliente, reduciendo el riesgo de modificar el carnet de otra persona.

1. Caja blanca de Mel — updateMine()

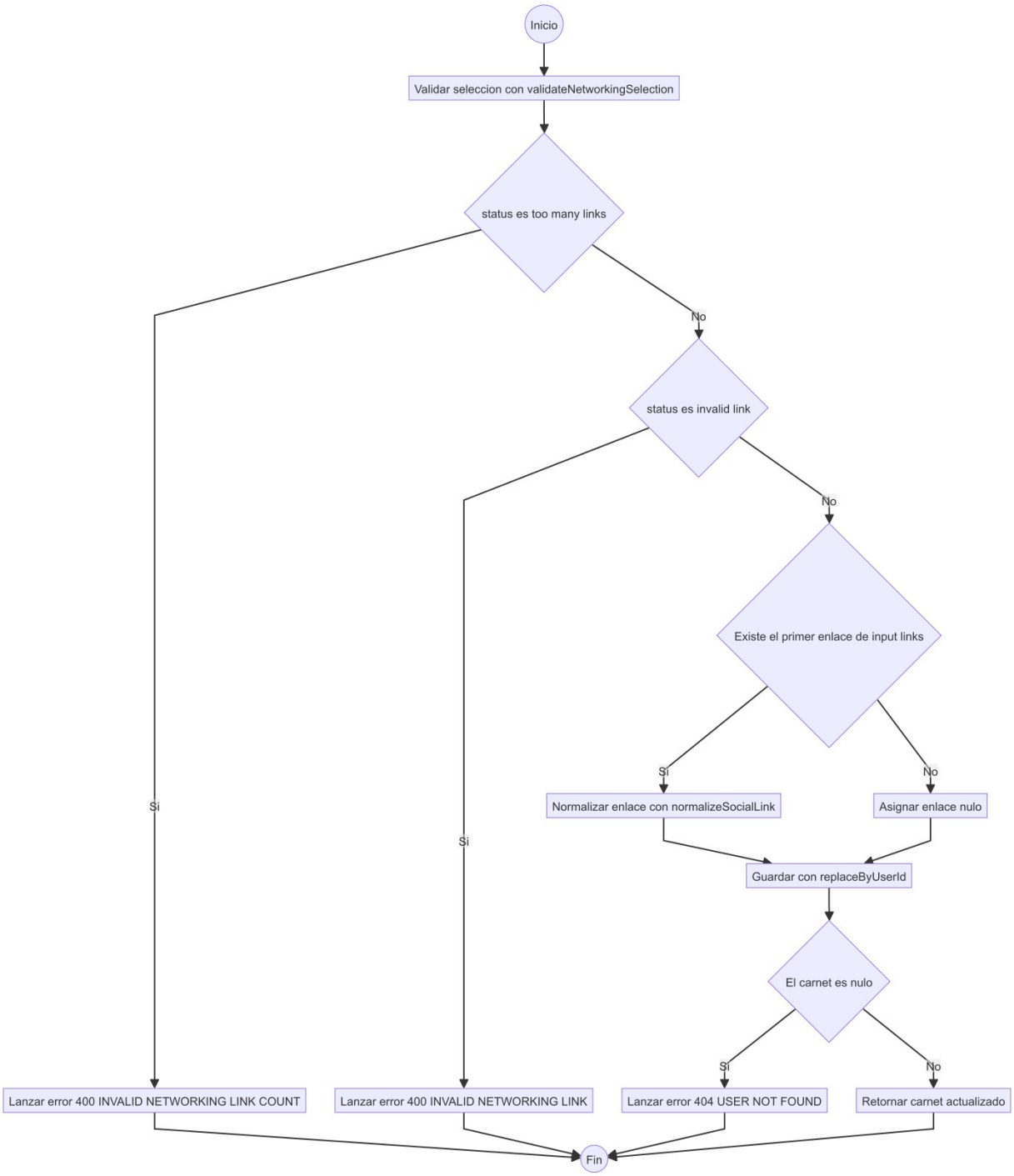
Producción: src/modules/networking/networking.service.ts:31–56
Prueba: test/HU25_mel/networking.cajablanca.test.ts

```
const validation = validateNetworkingSelection(input);
switch (validation.status) {
  case 'too_many_links': throw 400;
  case 'invalid_link': throw 400 con reason;
}
const link = input.links[0] ? normalizeSocialLink(input.links[0]) : null;
const card = await repository.replaceByUserId(userId, input.optIn, link);
if (!card) throw 404;
return card;
```

Complejidad y grafo

Decisión	Ramas
D1: too_many_links	Error 400 / continuar
D2: invalid_link	Error 400 con reason / continuar
D3: existe links[0]	normalizar enlace / usar null
D4: !card	404 / retornar carnet

```
V(G) = 4 decisiones + 1 = 5
Comprobación: E − N + 2 = 17 − 14 + 2 = 5
```



Grafo de updateMine. Las decisiones internas de validateNetworkingSelection no se suman porque la función medida es updateMine.

Archivo Mermaid editable: Guion_Pruebas_Jeff_Sam_Mel/mermaid/grafo_mel_update_mine.mmd

Cinco caminos

Camino	Recorrido	Oráculo
C1	validation=too_many_links	400 INVALID_NETWORKING_LINK_COUNT; 0 escrituras
C2	validation=invalid_link	400 INVALID_NETWORKING_LINK; reason=invalid_domain; 0 escrituras
C3	Enlace presente y válido	Trim de URL/label; persiste y retorna carnet
C4	links=[]	Persiste link=null; carnet sin red
C5	Repositorio devuelve null	404 USER_NOT_FOUND

TEXTO SUGERIDO PARA DECIR

“C1 y C2 comprueban que una entrada inválida se rechaza antes de escribir: el contador de llamadas permanece en cero. C3 es el camino feliz completo y permite mostrar cómo URL y label pierden espacios antes de persistirse.”

AAA del camino feliz

```
// Arrange: el repo captura userId, optIn y link
// Act
const result = await service.updateMine(42, {
  optIn: true,
  links: [{ platform: 'website', url: '  https://mel.dev  ', label: '  Web  ' }],
});
// Assert
expect(received.link).toEqual({ platform: 'website', url: 'https://mel.dev', label: 'Web' });
```

Mutación de Mel

```
bun run mut:mel
# muta: src/modules/networking/networking.service.ts:31-56
# oráculo: test/HU25_mel/networking.cajablanca.test.ts
```

Métrica	Resultado
Instrumentados	15
Killed	15
Survived	0
Mutation score	100% del rango

TEXTO SUGERIDO PARA DECIR

“La caja blanca detecta los quince cambios artificiales creados dentro de updateMine. No estoy afirmando que todo networking tenga cien por ciento; el alcance está explícito en stryker.mel.conf.json.”

2. Caja negra de Mel — updateNetworkingSchema

Producción: src/modules/networking/networking.schemas.ts
Prueba: test/HU25_mel/networking.cajanegra.test.ts

ALCANCE EXACTO

Se observa safeParse y el dato normalizado. No se prueba la ruta, el JWT, la transacción ni la BD.

Cinco campos y clases

Campo	Válido	Inválido / frontera
optIn	boolean	otro tipo; no hay caso directo actual
links	arreglo con 0 o 1 elemento	2 o más
platform	linkedin, instagram, github, x, website, other	valor fuera del enum
url	string trim, absoluta, http(s), ≤255, dominio coherente	vacía, relativa, ftp, >255, dominio ajeno
label	trim, ≤80; necesaria en website/other	vacía cuando requerida; >80

Ocho bloques, doce payloads

Bloque	Payload(s)	Esperado	Cantidad
CV1	optIn=true; links=[]	Válido	1

Bloque	Payload(s)	Esperado	Cantidad
CV2	GitHub válido con espacios	Válido y URL recortada	1
CNV1	Dos enlaces	Inválido: máximo 1	1
CNV2	ftp://github.com/a	Inválido: protocolo	1
CNV3	LinkedIn → example.com	Inválido: dominio	1
CNV4	website sin label / other con label	Inválido / válido	2
CNV5	platform fuera de enum / URL 260 / label 81	Los tres inválidos	3
CNV6	userId extra arriba / dentro del link	Ambos inválidos por strict	2

CNV5 contiene tres entradas y CNV6 dos. Por eso la forma precisa de describir la evidencia es ‘ocho bloques test, doce payloads y trece aserciones’.

TEXTO SUGERIDO PARA DECIR

“CNV6 es el caso de seguridad más importante: como el schema es estricto, rechaza userId tanto en el objeto superior como dentro del enlace. El usuario real siempre se toma del token autenticado.”

LÍMITES

La suite prueba URL de 260 (>255) y label de 81 (>80). Solo label cubre el valor inmediatamente posterior. No digas que se probaron ambos lados 255/256 para URL, porque no es cierto.

3. Cuatro unitarias de Mel

Producción: src/modules/networking/networking.logic.ts
Prueba: test/HU25_mel/networking.logic.test.ts

#	Función	Caso oficial	Qué protege
1	validateSocialLink	website/other sin etiqueta	Devuelve label_required
2	urlBelongsToPlatform	linkedin.com.evil.test / notgithub.com	Evita coincidencias por texto o path
3	validateNetworkingSelection	Dos enlaces	Devuelve too_many_links
4	normalizeSocialLink	Espacios en URL/label	Trim; label vacío → null

CONTEO REAL

El archivo completo prueba cinco funciones mediante nueve bloques y 23 aserciones. isHttpUrl es una función de apoyo; para la rúbrica se exponen las cuatro funciones de la captura.

Cómo se valida el dominio

```
hostname === expected || hostname.endsWith(`.${expected}`)
```

- linkedin.com es válido por igualdad exacta.
- pe.linkedin.com es válido porque termina en .linkedin.com.
- linkedin.com.evil.test es inválido: el dominio registrable termina en evil.test.
- notgithub.com es inválido: no existe un punto antes de github.com.
- Un texto instagram.com colocado en el path no cambia el hostname.
- La plataforma x acepta x.com y twitter.com como alias declarados.

Normalización y selección

```
if (input.links.length > 1) return too_many_links;
const link = input.links[0];
if (!link) return ok;
return validateSocialLink(link);

normalizeSocialLink(link) => {
  platform,
  url: link.url.trim(),
  label: trimmedLabel.length > 0 ? trimmedLabel : null,
}
```

Preguntas probables para Mel

Pregunta	Respuesta breve y segura
¿Por qué validar en Zod y servicio?	Zod protege HTTP; el servicio conserva invariantes si se llama desde otra entrada.
¿Por qué no acepta notgithub.com?	Se exige igualdad o sufijo .github.com; una coincidencia textual no basta.
¿Por qué website necesita label?	No tiene una marca reconocible; necesita un nombre visible como Portafolio.
¿Qué significa null del repositorio?	Que el propietario autenticado no existe; se traduce a 404.
¿Por qué userId no va en body?	Se obtiene del JWT para impedir que el cliente elija otro propietario.
¿0 enlaces con optIn=true es válido?	Sí en el código ejecutable actual; se comparte el carnet sin red.
¿8 casos o 12?	Ocho bloques test que contienen doce payloads concretos.
¿15/15 cubre toda HU25?	No; solo updateMine en el rango 31–56.

DISCREPANCIA ESPECIFICACIÓN/CÓDIGO

La especificación escrita indica que optIn=true exigiría exactamente un enlace, pero schema, lógica y pruebas actuales aceptan cero o uno. En esta exposición describan el comportamiento ejecutable y, si preguntan, reconozcan la discrepancia como deuda de alineación.

Demo de Mel

```
cd ~/ULIMA++/ULima_Backend_IS2

bun test test/HU25_mel/networking.cajablanca.test.ts
bun test test/HU25_mel/networking.cajanegra.test.ts
bun test test/HU25_mel/networking.logic.test.ts
bun run mut:mel
```

Versión de 2 minutos de Mel

TEXTO SUGERIDO PARA DECIR

“Mi caja blanca es updateMine: dos salidas del switch, el ternario enlace/null y el repositorio null producen V(G)=5 y cinco caminos; Stryker mata 15 de 15 mutantes del rango. Mi caja negra trata el schema Zod como contrato observable con cinco campos, ocho bloques y doce payloads, incluido modo estricto contra userId extra. Mis cuatro funciones unitarias validan etiqueta, pertenencia de dominio, máximo una red y normalización de URL/label.”

Errores que Mel debe evitar

- Decir que la caja negra usa base de datos o endpoint completo.
- Afirmar que hay exactamente ocho payloads: hay ocho bloques y doce payloads.
- Incluir isHttpUrl como quinta unitaria oficial sin explicar que es apoyo.
- Decir que URL 255/256 está cubierta bilateralmente.
- Extender el 15/15 a todo networking o a todo el proyecto.

Anexos para los tres

A. Comandos de ejecución independiente

```
cd ~/ULIMA++/ULima_Backend_IS2

# JEFF
bun test test/HU01_jeff/login.cajablanca.test.ts
bun test test/HU18_jeff/advising.cajanegra.test.ts
bun test test/HU20_jeff/password-reset.logic.test.ts
bun run mut:jeff

# SAM
bun test test/HU30_sam/notificar.cajablanca.test.ts
bun test test/HU22_sam/impedidos.cajanegra.test.ts
bun test test/HU22_sam/impedidos.unitarias.test.ts test/HU06_sam/grades.logic.test.ts
bun run mut:sam

# MEL
bun test test/HU25_mel/networking.cajablanca.test.ts
bun test test/HU25_mel/networking.cajanegra.test.ts
bun test test/HU25_mel/networking.logic.test.ts
bun run mut:mel

# CONTROL GLOBAL
bun run build
```

B. Qué mostrar en VS Code

1. Abrir producción a la izquierda y test a la derecha.
2. Buscar ´ en el test para ir al comentario central.
3. Señalar primero la decisión de producción y luego el caso que la fuerza.
4. Mostrar Arrange, Act y Assert de un caso feliz y un caso de error.
5. Abrir el Mermaid en preview solo después de que el flujo funcional esté claro.
6. Ejecutar Bun; luego Stryker solo si hay tiempo suficiente.

C. Checklist antes de exponer

✓	Comprobación
☐	Cada persona sabe sus tres archivos y no abre carpetas antiguas.
☐	Los tres pueden definir caja blanca, negra, unitaria y mutante en una frase.
☐	Jeff dice V(G)≈8; Sam V(G)=10; Mel V(G)=5.
☐	Sam distingue 147/147 de asistencia y 156/156 total puntuable.
☐	Mel distingue 8 bloques de 12 payloads.
☐	Todos dicen que mutation score tiene alcance limitado.
☐	Los comandos se ejecutan desde ULima_Backend_IS2.
☐	Se cerraron procesos pesados y se aumentó el tamaño del terminal.
☐	Hay capturas/reportes como plan B si una demo tarda.

D. Preguntas transversales

Pregunta	Respuesta modelo
¿Por qué no usar BD real?	Porque una prueba unitaria/caja blanca necesita aislar decisiones; la BD introduciría estado y latencia.
¿Un stub invalida la prueba?	No. Se sustituye la dependencia, pero se ejecuta la unidad real y se observan sus salidas/interacciones.
¿Qué es un oráculo?	La regla que decide si el resultado observado coincide con el esperado.

Pregunta	Respuesta modelo
¿Por qué probar fronteras?	Los errores de comparación suelen aparecer exactamente en < frente a ≤, máximos y valores vacíos.
¿Qué mata un mutante?	Al menos una prueba falla al ejecutarse contra el código alterado.
¿Qué es un mutante equivalente?	Un cambio sintáctico que no modifica ninguna salida observable; no existe un test capaz de distinguirlo.
¿Pasar pruebas demuestra que no hay bugs?	No; demuestra que el comportamiento cubierto coincide con los oráculos definidos.
¿Por qué comentarios en tests?	Documentan intención, camino, partición y razón de cada aserción sin alterar producción.

E. Evidencia local y portabilidad

REPORTES HTML

reports/mutation está ignorado por Git. Al compartir solo el repositorio, otra persona debe regenerar los reportes con bun run mut:jeff, bun run mut:sam o bun run mut:mel. Este Word sí conserva las cifras verificadas y los grafos incrustados.

Los archivos Mermaid editables se encuentran junto a este documento en la carpeta mermaid. Los tres usan sintaxis flowchart TD y pueden abrirse con una extensión Mermaid de VS Code.

F. Cierre conjunto sugerido

TEXTO SUGERIDO PARA DECIR

“En conjunto usamos caja blanca para recorrer decisiones internas, caja negra para validar contratos mediante particiones y fronteras, y pruebas unitarias para aislar reglas puras. Las 98 pruebas seleccionadas pasan, TypeScript compila y los tres rangos de mutación no tienen sobrevivientes válidos. Interpretamos esa evidencia dentro de su alcance y dejamos explícitas las limitaciones encontradas.”

Fin del guion

Documento generado desde el estado real del workspace. Fecha de verificación: 17 de julio de 2026 (America/Lima).